

INFOSYS 720 – Information Systems Research
Assignment 4: Design – Information Systems Artefacts

Aidil Muslim
463734221

Title:

**Task-Aware Prompt Calibration for Generating Synthetic Novice
Python Code**

Paper Template:

Association for Computing Machinery (ACM) - SIG Proceedings
Template

Task-Aware Prompt Calibration for Generating Synthetic Novice Python Code

Aidil Muslim

amus790@aucklanduni.ac.nz

University of Auckland

Auckland, New Zealand

ABSTRACT

Large language models offer a potential source of synthetic code for evaluating educational analysis and feedback systems, but generated programs may not reproduce the task-sensitive stylistic and structural defects observed in authentic novice submissions. This design phase develops two information-system artefacts for task-aware generation of functionally correct synthetic novice Python code. The first, a Context-Aware Defect-Informed Generation Architecture, separates task-independent defect tendencies from task-dependent defects selected according to programming-task context and composes them into a target profile. The second, a Synthetic Novice Code Generation and Calibration Prototype, operationalises that profile through generation specification, LLM-based code generation, functional validation, defect detection, profile comparison, and iterative calibration. An executable Python/Streamlit proof of concept demonstrates the feasibility of the integrated workflow. Together, the artefacts provide a traceable system design for later empirical evaluation using defect profiles derived from authentic CS1 submissions.

CCS Concepts

• **Social and professional topics~ Professional topics~ Computing education~ Computing education programs~ Computer science education~ CS1**

Keywords

novice programming; code style defects; synthetic data; AI code generation; prompt engineering

1. Proposed System Artefacts

1.1 System Design Overview

The preceding research phase established conceptual artefacts for representing novice code-quality characteristics according to programming context and for using those representations to guide synthetic code generation. The Context-Aware Synthetic Representation Framework distinguishes between characteristics that remain comparatively stable across tasks and those whose occurrence varies with task context. The Iteratively Calibrated Synthetic Generation Framework further specifies how the resulting target representation can inform generation and be refined according to discrepancies between intended and generated characteristics. This emphasis on correspondence between the characteristics of an observed domain and their

information-system representation is consistent with Representation Theory, which considers representational faithfulness an important property of information systems (Recker et al., 2019; Wand & Weber, 1988).

The present design phase translates these conceptual artefacts into two complementary system artefacts. The first, a Context-Aware Defect-Informed Generation Architecture, defines how task-independent and task-dependent defect tendencies are represented and incorporated into generation for a given programming task. The second, a Synthetic Novice Code Generation and Calibration Prototype, instantiates this architecture through an executable workflow comprising prompt construction, code generation, functional validation, defect analysis, profile comparison, and iterative refinement. Developing and instantiating these artefacts follows the design-science view of systems development as a research activity in which purposeful artefacts are constructed to address identified problems and requirements (Hevner et al., 2004; Nunamaker et al., 1990).

Positioning the system against these foundations clarifies the design contribution. Representation Theory provides the criterion of correspondence between authentic domain phenomena and their information-system representation. The defect catalogue developed by Řechtáčková, Pelánek and Effenberger (2024) provides the vocabulary for specifying the novice-code characteristics to be represented, while (Leinonen et al., 2025) demonstrate that synthetic student-code distributions can be compared with authentic submissions. The proposed system artefacts extend these foundations by making defect representation task-aware and by incorporating observed profile discrepancies into iterative generation calibration. The contribution therefore lies in integrating and operationalising these foundations rather than replacing them.

1.2 Artefact 1: Context-Aware Defect-Informed Generation Architecture

The first system artefact is a Context-Aware Defect-Informed Generation Architecture that specifies how novice code-quality defect tendencies are incorporated into synthetic code generation according to programming-task context. The architecture avoids applying a single aggregated defect profile uniformly across tasks, since the occurrence of novice code-quality defects can vary across programming topics and constructs (Effenberger & Pelánek, 2022; Řechtáčková et al., 2025). Instead, it distinguishes general defect tendencies from those whose relevance depends on the task being represented.

1.2.1 Purpose and Design Principles

The architecture preserves the distinction between task-independent and task-dependent defect tendencies. Task-independent defects are characteristics whose occurrence remains comparatively stable across programming tasks, whereas task-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee, provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

INFOSYS720, The University of Auckland

© 2026 Copyright held by the owner/author(s)

dependent defects vary according to the requirements or constructs associated with a particular task.

The central design principle is therefore that task-independent defects remain eligible across programming contexts and are

represented according to their target prevalence, whereas task-dependent defects are first selected according to task applicability

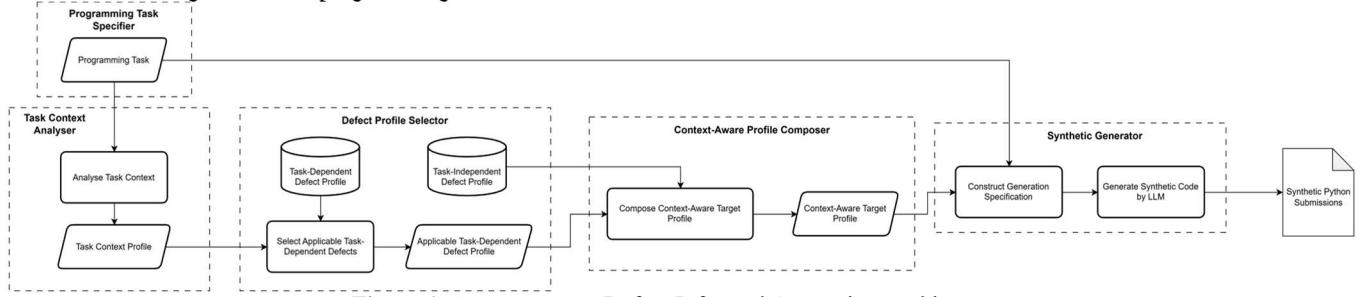


Figure 1. Context-Aware Defect-Informed Generation Architecture

and subsequently represented according to their task-specific prevalence. This distinction prevents context-specific characteristics, such as defects associated with branching or iteration, from being introduced into tasks for which the corresponding programming construct is not relevant. Task context is consequently treated as an explicit part of the synthetic representation rather than being left entirely to the language model.

1.2.2 Architecture Components

The architecture contains six principal components: Programming Task Specifier, Task Context Analyser, Task-Independent Defect Profile, Task-Dependent Defect Profile, Context-Aware Profile Composer, and Synthetic Generator. Their roles are summarised in Table 1.

Table 1. Context-Aware Defect-Informed Generation Architecture components and responsibilities

Component	Responsibility
Programming Task Specifier	Specifies the functional programming requirements
Task Context Analyser	Represents predefined programming-task characteristics
Task-Independent Defect Profile	Stores broadly applicable defect tendencies and target prevalence
Task-Dependent Defect Profile	Stores context-linked defect tendencies and task-specific prevalence
Context-Aware Profile Composer	Constructs the target profile for the selected task
Synthetic Generator	Translates the task and target profile into synthetic generation submissions.

The Programming Task Specifier provides the functional problem to be solved. The Task Context Analyser identifies relevant features of the task, such as whether the expected solution involves conditional branching, iteration, collection processing, input validation, or decomposition. These characteristics provide the context required to determine which task-dependent defects are appropriate for the current generation task.

The Task-Independent Defect Profile follows a separate information path because its characteristics are not primarily determined by a specific task context. In contrast, the Task-Dependent Defect Profile is filtered according to the characteristics identified for the selected programming task. The two sources are subsequently integrated by the Context-Aware

Profile Composer before generation constraints are constructed. The conceptual design is shown in Figure 1.

1.2.3 Context-Aware Profile Construction

The architecture separates defect applicability from defect prevalence. Task-independent defects remain eligible across tasks, with their target prevalence specifying the intended occurrence across a collection of synthetic submissions. Task-dependent defects undergo an additional contextual selection step: the Task Context Profile determines which task-dependent defects are applicable before its task-specific prevalence is incorporated.

For a programming task T , the Context-Aware Target Profile can be represented as:

$$P_T = C(P_I, P_D(T))$$

where P_T denotes the Context-Aware Target Profile for task T , P_I represents the task-independent defect profile, and $P_D(T)$ represents the task-dependent defect profile applicable to task T , and C denotes the context-aware profile-composition process. The resulting profile preserves the classification and target prevalence associated with each defect rather than combining the two sources as an undifferentiated set.

For the proof of concept, task contexts and profile values can be predefined demonstration inputs. These inputs demonstrate the architecture without being interpreted as empirical findings; subsequent empirical analysis can replace them with classifications and prevalence estimates derived from authentic CS1 submissions.

1.2.4 Generation Behaviour

The Context-Aware Target profile is supplied to the Synthetic Generator, which constructs a Generation Specification while maintaining a separation between the functional task requirements and the defect characteristics to be represented. This relationship can be expressed as:

$$G(T) = F(T, P_T)$$

where $G(T)$ denotes the generation specification for the task T , T represents the functional programming-task requirements, P_T represents the corresponding Context-Aware Target Profile, and F denotes the profile-to-prompt construction process.

Inclusion of a defect in P_T does not require it to occur in every generated program. Instead, its prevalence specifies its intended representation across the generated collection. For example, two tasks may share task-independent tendencies relating to naming or unnecessary intermediate variables. A conditional-validation task may additionally activate defects associated with Boolean expressions or branching, whereas a list-processing task may

activate defects associated with iteration or collection manipulation. The architecture determines both which characteristics are appropriate for a task and the prevalence with which they should be represented, providing the contextual generation target used by the subsequent proof-of-concept artefact.

1.3 Artefact 2: Synthetic Novice Code Generation and Calibration Prototype

The second system artefact is a Synthetic Novice Code Generation and Calibration Prototype that operationalises the Context-Aware Target Profile produced by the first artefact. As illustrated in Figure 2, the target profile is translated into a generation specification that guides the LLM in producing synthetic Python submissions. The prototype subsequently evaluates the generated code for both functional correctness and correspondence with the intended defect profile.

Generated code is first assessed for functional correctness. If a submission fails the functional requirements, the resulting functional-test feedback is incorporated into the reconstructed generation specification before another generation attempt. Separating functional validation from defect analysis ensures that the intended novice-style characteristics can be evaluated without treating functional failure as a desired defect. Functionally correct submissions proceed to defect detection, where the targeted characteristics are identified and aggregated into an Observed Synthetic Defect Profile.

The observed defect profile is then compared with the Context-Aware Target Profile. Distribution-level comparison is used because the intended representation concerns defect tendencies across a collection of submissions rather than the realism of individual examples, consistent with prior work comparing authentic and synthetic programming-submission distributions (Leinonen et al., 2025). If the discrepancy falls within the predefined tolerance, the generated programs are retained as the synthetic Python submissions. Otherwise, the relevant generation constraints are calibrated according to the observed discrepancy and incorporated into a reconstructed Generation Specification before another generation cycle.

1.4 Integration of the Artefacts

The proposed system integrates the context-aware generation architecture and calibration prototype into a single end-to-end process, as illustrated in Figure 3. The programming task is first analysed to establish its task context, which guides the selection of applicable task-dependent defects. These characteristics are combined with the task-independent defect profile to construct the Context-Aware Target Profile, providing the representational target for synthetic generation.

The programming task and target profile are subsequently translated into a generation specification and supplied to the LLM. Generated code is first assessed for functional correctness; unsuccessful outputs produce functional feedback that informs reconstruction of the generation specification. Functionally correct submissions proceed to defect detection, where the observed defect profile is compared with the target profile. Where the discrepancy exceeds the predefined tolerance, the relevant generation constraints are calibrated and incorporated into a revised specification. Otherwise, the generated programs are retained as the final synthetic Python submissions.

Together, the integrated artefacts establish a continuous process from task-context identification and target-profile construction to generation, validation, comparison, and iterative refinement.

2. Novelty and Originality

The novelty of the proposed system artefacts lies in how existing knowledge and technologies are combined to support context-aware generation and iterative calibration of synthetic novice Python code. The literature review established that prior research separately provides evidence on authentic novice-code characteristics, LLM-based code generation, prompt control, synthetic student-code generation, and evaluation methods. However, limited work integrates these elements into a process that distinguishes defect tendencies according to task dependence, incorporates the resulting profiles into generation, and subsequently evaluates generated code against the intended defect distribution. The proposed artefacts address this intersection through the Context-Aware Defect-Informed Generation Architecture and the Synthetic Novice Code Generation and Calibration Prototype.

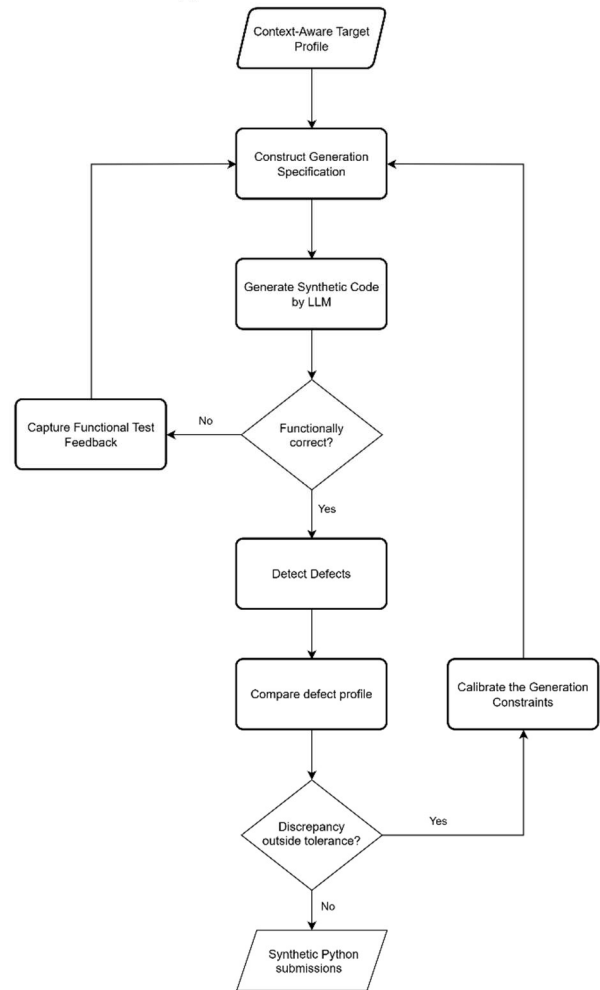


Figure 2. Synthetic Novice Code Generation and Calibration Prototype

2.1 Comparison with Existing Research

Research on novice programming demonstrates that student submissions exhibit recurring stylistic, structural, and code-quality characteristics beyond functional correctness, and that the prevalence of such characteristics can vary across programming topics and constructs (Effenberger & Pelánek, 2022; Řečtáčková et al., 2024, 2025). Other studies demonstrate that LLMs can generate introductory-programming solutions and that generated

outputs can be influenced through prompt formulation and contextual guidance (Denny et al., 2023; Finnie-Ansley et al., 2022; Reeves et al., 2023). More directly, synthetic

programming-data research has shown that LLMs can generate student-like or deliberately defective submissions and that distributions of generated outputs can be compared with authentic

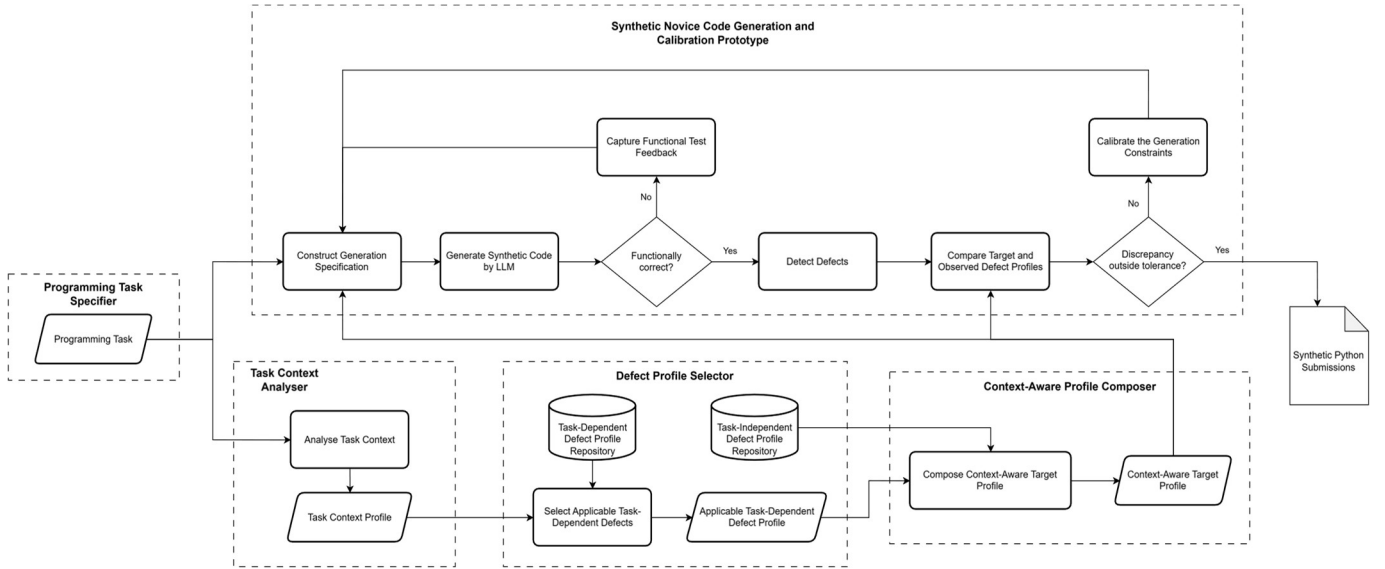


Figure 3. Integrated Architecture of the Context-Aware Synthetic Novice Code Generation and Calibration System

student data (Denny et al., 2026; Leinonen et al., 2025; Pădurean et al., 2026).

The proposed artefacts extend these research streams in two ways. First, the generation architecture explicitly separates task-independent from task-dependent defect tendencies and uses programming-task context to determine which task-dependent characteristics should contribute to generation. This differs from treating novice characteristics as a single aggregate profile. Second, the prototype incorporates distributional comparison into the generation process itself. Rather than using target-versus-generated comparison only as a final evaluation, observed discrepancies are used to revise the generation specification and initiate another generation cycle.

2.2 Comparison with Existing Systems

The systematic systems review found that the technological capabilities required by the proposed system already exist, but are distributed across separate technologies. LLM platforms such as Google Gemini API, Hugging Face Inference Providers, and Ollama support programmable code generation and prompt control. PrairieLearn and pytest support automated functional validation, while Python AST and Ruff provide mechanisms for structural and rule-based code analysis. None of the reviewed systems, however, provides an integrated mechanism that begins with a context-aware novice-defect profile, translates it into generation constraints, evaluates the resulting programs for both functional correctness and defect-profile correspondence, and uses discrepancies to refine subsequent generation.

2.3 Nature of the Contribution

Following Gregor & Hevner (2013) design-science contribution framework, the proposed artefacts are positioned primarily as an improvement, with an exaptive element. The design improves existing approaches by integrating established generation, testing, code-analysis, and distributional-comparison capabilities into a coherent process centred on context-aware defect profiles.

The exaptive element arises from applying established mechanisms, particularly static code analysis, distributional comparison, and iterative prompt refinement, to the specific objective of reproducing task-sensitive novice-code characteristics. Therefore, the originality of the contribution lies in the context-aware separation of defect tendencies and their integration with iterative generation and calibration, providing a system design that more directly connects empirical novice-code representation with controllable synthetic-code production.

3. Design Rationale

The proposed system artefacts were designed to maintain explicit traceability between the identified research problems, the resulting system requirements, and the conceptual mechanisms developed to address them. The central design problem concerns the generation of functionally correct synthetic novice code whose observable defect characteristics correspond to those found in authentic CS1 submissions. Addressing this problem requires more than combining an LLM with code-analysis tools. The system must determine which characteristics are appropriate for a given programming task, translate those characteristics into generation constraints, evaluate the resulting code against both functional and representational criteria, and refine generation when the resulting defect distribution diverges from the intended target. These requirements provide the rationale for the principal components and relationships within the proposed architecture.

3.1 Artefact Traceability

Table 3 summarises the progression from the identified issues and requirements to the conceptual and system artefacts. The traceability is intended to minimise unexplained design choices by demonstrating how each major system feature addresses a specific requirement.

The resulting design therefore follows a progression from characteristic classification, through context-aware representation, to controlled generation and refinement. This progression is also consistent with Representation Theory, as the

target profile represents selected observable properties of authentic novice submissions, while the generated programs constitute synthetic representations whose correspondence with

those properties can subsequently be assessed (Recker et al., 2019).

Table 2. Traceability between research issues, requirements, conceptual artefacts, and system features

Problem/Issue	Requirement	Conceptual Artefact	System Feature
Aggregate defect profiles may obscure variation across programming tasks.	Distinguish task-independent and task-dependent defect tendencies.	Context-Aware Characteristic Classification Framework	Separate Task-Independent Defect Profile and Task-Dependent Defect Profile
Task-dependent defects vary according to programming context.	Construct a Context-Aware Target Profile for the selected task.	Context-Aware Synthetic Representation Framework	Task Context Analyser, applicable-defect selection, and Context-Aware Profile Composer
Defect profiles are not systematically translated into generation constraints.	Support profile-informed synthetic generation.	Iteratively Calibrated Synthetic Generation Framework	Generation Specification constructed from the programming task and Context-Aware Target Profile
Functional correctness and defect correspondence represent different properties of generated code.	Evaluate functional validity and defect representation separately.	Iteratively Calibrated Synthetic Generation Framework	Functional Validation, Defect Detection, and profile comparison.
Generated defect distributions may diverge from the intended target.	Support iterative refinement of generation.	Iteratively Calibrated Synthetic Generation Framework	Tolerance assessment and calibration of generation constraints
Generation, testing, defect analysis, and refinement are technologically separated.	Integrate the required capabilities within a coherent process.	Integrated conceptual contribution	Synthetic Novice Code Generation and Calibration Prototype

3.2 Design Decisions and Justification

A principal design decision is the separation of task-independent and task-dependent defect information. Existing evidence indicates that code-quality defects are not uniformly distributed across introductory programming contexts (Effenberger & Pelánek, 2022; Řečtáčková et al., 2025). Combining these tendencies into a single aggregate profile could therefore obscure contextual variation or introduce defects into tasks for which they are not meaningful. The architecture instead preserves task-independent tendencies as general characteristics while using task context to determine which task-dependent defects contribute to the target profile.

This distinction motivates the inclusion of the Task Context Analyser. Task-dependent defects cannot be incorporated solely on the basis of their overall prevalence; their applicability must first be determined from the programming task being represented. The Task Context Analyser therefore mediates between the Programming Task and the Task-Dependent Defect Repository, while the Task-Independent Defect Profile remains available independently of this contextual selection. The Context-Aware Profile Composer subsequently integrates the two sources into a task-specific representational target.

A further design decision is to maintain a separation between the Context-Aware Target Profile and the Generation Specification. The target profile specifies the characteristics that the synthetic collection is intended to reproduce, whereas the generation specification expresses how those characteristics are communicated to the LLM. This separation is necessary for iterative calibration: discrepancies in generated outputs should modify the instructions used to approximate the target rather than alter the target itself.

Functional validation is similarly separated from defect-profile assessment. The research concerns stylistic and structural

characteristics that may occur in otherwise functionally correct programs; consequently, functional failure cannot be treated as evidence of successful novice-code representation. Functionally valid outputs therefore proceed to defect detection and profile comparison as a distinct evaluation stage.

Finally, both functional-test feedback and defect-profile discrepancy are routed back through the Generation Specification. Evaluation does not directly modify generated programs. Instead, it informs revised generation instructions, which guide the subsequent LLM generation cycle. This design preserves the programming task and representational target while allowing the generation mechanism to adapt iteratively to observed failures and discrepancies.

4. Technical Soundness and Feasibility

The technical feasibility of the proposed system depends on whether its principal functions can be implemented using compatible and reproducible technologies. These functions include context-aware profile representation, generation-specification construction, LLM-based code generation, functional validation, defect detection, profile comparison, and iterative calibration. Existing technologies already support these individual capabilities, including programmatic LLM access, automated Python testing, and structural source-code analysis. Thus, the technical challenge concerns their integration according to the information flow and feedback mechanisms defined by the proposed system architecture.

4.1 Technology Stack

The proof of concept is designed as a Python-based application so that generation control, testing, source-code analysis, profile processing, and visualisation can be implemented within a common programming environment. A lightweight Streamlit

interface provides the user-facing layer for selecting a programming task, inspecting the Context-Aware Target Profile, initiating generation, and reviewing evaluation outcomes.

LLM generation can be implemented through a programmable model interface such as the Google Gemini API, which supports programmatic generation and system-level instructions. The generation component is conceptually separated from the remaining application logic so that alternative hosted or local models can subsequently be substituted without redesigning the profile-construction and evaluation components.

Functional correctness can be assessed through predefined test cases using pytest. Generated Python source code can subsequently be parsed using Python's built-in ast module, enabling selected structural characteristics to be identified through explicit detection rules. Ruff may provide supplementary static-analysis capabilities where its existing rules correspond to the defect categories represented by the prototype (Astral Software, n.d.; Google, n.d.; pytest Development Team, n.d.; Python Software Foundation, n.d.). The resulting stack therefore provides technically compatible components for implementing the principal functions identified in the system architecture.

4.2 Prototype Processing Logic

The executable prototype follows the integrated workflow presented in Figure 3. The Programming Task and Context-Aware Target Profile form the principal inputs to the generation process. These inputs are translated into a Generation Specification containing the functional requirements of the programming task and the novice-code characteristics intended to be represented.

The Generation Specification is supplied to the LLM to produce synthetic Python code. Each generated submission is subsequently evaluated against predefined functional tests. Where a submission fails the functional requirements, the resulting test feedback is incorporated into the reconstruction of the Generation Specification before another generation attempt. Functionally correct submissions proceed to defect detection, where the selected target characteristics are identified and recorded.

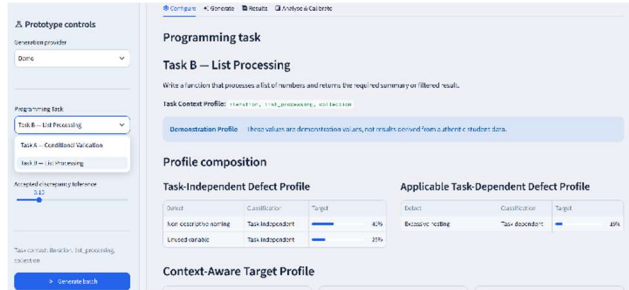


Figure 4. Prototype interface for task selection, context-aware target-profile configuration, and synthetic-code generation

Figure 4 presents the prototype configuration interface through which the programming task and applicable defect characteristics are inspected before generation. The interface makes the task-independent and task-dependent contributions to the Context-Aware Target Profile explicit and provides the control through which a generation cycle is initiated.

Figure 5 demonstrates the subsequent processing of generated submissions. The interface exposes the generated source code together with functional-validation outcomes and detected defect characteristics, providing observable evidence that functional correctness and defect representation are evaluated as distinct properties.

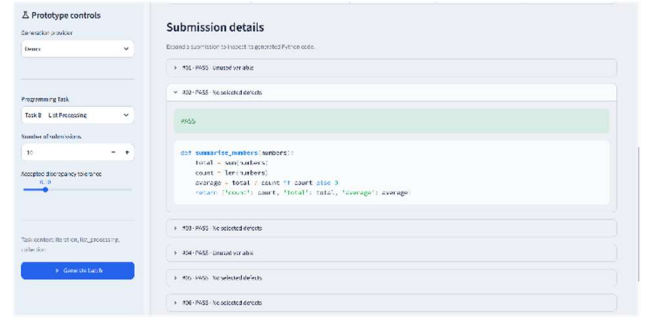


Figure 5. Prototype output showing generated submissions, functional-validation results, and detected defect characteristics

4.3 Defect Detection

The proof of concept is designed around a restricted set of defect categories that can be detected using transparent and reproducible rules. This restriction is appropriate because the objective of the prototype is to demonstrate the feasibility of the generation and calibration mechanism rather than implement the complete novice-code defect catalogue.

Generated programs are parsed into abstract syntax trees and traversed to identify selected stylistic or structural patterns. Candidate categories include redundant Boolean comparisons, unnecessary variables, selected naming patterns, and excessive structural nesting, provided that each category can be consistently mapped to the adopted defect taxonomy. Each detector records the occurrence of a targeted characteristic within an individual submission. These results are then aggregated across the functionally valid submissions to construct the Observed Synthetic Defect Profile.

Rule-based detection also supports reproducibility because the conditions used to classify each defect remain explicit and can be independently inspected or revised.

4.4 Calibration Mechanism

Calibration is driven by the discrepancy between the Context-Aware Target Profile and the Observed Synthetic Defect Profile. For defect category d , the discrepancy can be represented as:

$$e_d = T_d - O_d$$

where T_d denotes the target prevalence and O_d denotes the observed prevalence among functionally valid synthetic submissions.

A predefined tolerance determines whether further refinement is required. Where the discrepancy remains within this tolerance, the generated distribution is considered sufficiently aligned with the target. Where a characteristic is underrepresented or overrepresented, the corresponding generation constraint is adjusted before the Generation Specification is reconstructed.

The target profile itself remains unchanged during calibration. This distinction is technically important because the target profile represents the intended synthetic characteristics, whereas the Generation Specification represents the mechanism through which the LLM attempts to reproduce them.

Figure 6 presents the comparison between target and observed defect prevalence and the corresponding calibration outcomes. The visualisation makes profile discrepancy observable and identifies the generation constraints requiring maintenance or adjustment before a subsequent generation cycle.

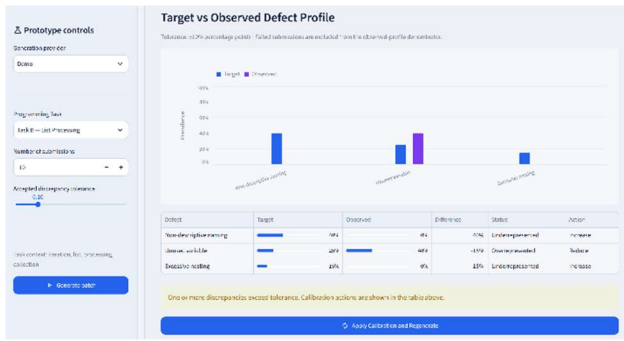


Figure 6. Target-observed defect-profile comparison and resulting calibration actions

In the illustrated scenario, non-descriptive naming is underrepresented relative to its target, whereas excessive nesting is overrepresented; the prototype consequently recommends increasing the former constraint and reducing the latter, while characteristics within tolerance are maintained.

4.5 Prototype Evidence and Feasibility

Figures 4–6 collectively demonstrate the executable instantiation of the proposed architecture. They show the configuration of the Programming Task and Context-Aware Target Profile, functional validation and defect analysis of generated submissions, and comparison of the resulting defect distribution with the intended target for subsequent calibration.

Successful execution of this workflow demonstrates that the two system artefacts can operate as an integrated generation–evaluation–refinement process, providing evidence of technical feasibility through executable instantiation (Nunamaker et al., 1990).

4.6 Technical Limitations

The proof of concept is intentionally narrower than the complete empirical system. It will initially support a limited number of programming tasks, a selected set of automatically detectable defect categories, and demonstration target profiles. Consequently, the prototype demonstrates technical feasibility rather than establishing the empirical accuracy of the generated distributions.

Execution of generated Python code also requires appropriate safeguards, including execution timeouts, resource restrictions, and isolation from the host environment. Broader defect coverage, alternative LLM providers, stronger execution isolation, and empirically derived target profiles can subsequently be incorporated without changing the principal system architecture.

4.7 Prototype Development GitHub Project

The prototype project is stored in GitHub. The repository is temporarily in public visibility. The repository can be accessed through this link: <https://github.com/ADI2L-M/synthetic-code-prototype>

References

- Astral Software. (n.d.). *Rules*. Retrieved August 12, 2026, from <https://docs.astral.sh/ruff/rules/>
- Denny, P., Kumar, V., & Giacaman, N. (2023). Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1, SIGCSE 2023*, 1136–1142. <https://doi.org/10.1145/3545945.3569823>
- Denny, P., Riegel, K., Luxton-Reilly, A., Leinonen, J., Prather, J., & MacNeil, S. (2026). Enabling Postgraduate Projects in Computing Education through Synthetic Research Data Generation. *Proceedings of the 28th Australasian Computing Education Conference, ACE '26*, 85–94. <https://doi.org/10.1145/3786228.3786241>
- Effenberger, T., & Pelánek, R. (2022). Code Quality Defects across Introductory Programming Topics. *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education (SIGCSE 2022)*. <https://doi.org/10.1145/3478431.3499415>
- Finnie-Ansley, J., Denny, P., Becker, B. A., Luxton-Reilly, A., & Prather, J. (2022). The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. *Proceedings of the 24th Australasian Computing Education Conference, ACE '22*, 10–19. <https://doi.org/10.1145/3511861.3511863>
- Google. (n.d.). *Text Generation*. Retrieved August 12, 2026, from https://ai.google.dev/gemini-api/docs/text-generation#python_7
- Gregor, S., & Hevner, A. R. (2013). Positioning and Presenting Design Science Research for Maximum Impact. *MIS Quarterly*, 37(2), 337–355. <https://doi.org/10.25300/MISQ/2013/37.2.01>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Leinonen, J., Denny, P., Kiljunen, O., MacNeil, S., Sarsa, S., & Hellas, A. (2025). LLM-itation is the Sincerest Form of Data: Generating Synthetic Buggy Code Submissions for Computing Education. *Proceedings of the 27th Australasian Computing Education Conference, ACE '25*, 56–63. <https://doi.org/10.1145/3716640.3716647>
- Nunamaker, J. F., Chen, M., & Purdin, T. D. M. (1990). Systems Development in Information Systems Research. *Journal of Management Information Systems*, 7(3), 89–106. <https://doi.org/10.1080/07421222.1990.11517898>
- Pădurean, V.-A., Riegel, K., Gotovos, A., Mahapatra, J., Ghosh, A., Denny, P., Leinonen, J., Prather, J., & Singla, A. (2026). *When AI Is Wrong on Purpose: How Students Respond to Buggy GenAI Code*. <https://doi.org/10.48550/arXiv.2607.05068>
- pytest Development Team. (n.d.). *How to parametrize fixtures and test functions*. Retrieved August 12, 2026, from <https://docs.pytest.org/en/stable/how-to/parametrize.html>
- Python Software Foundation. (n.d.). *ast — Abstract syntax trees*. Retrieved August 12, 2026, from <https://docs.python.org/3/library/ast.html>
- Řečtáčková, A., Pelánek, R., & Effenberger, T. (2024). Catalog of Code Quality Defects in Introductory Programming. *Proceedings of the 2024 Innovation and Technology in Computer Science Education Conference (ITiCSE 2024)*. <https://doi.org/10.1145/3649217.3653638>
- Řečtáčková, A., Pelánek, R., & Effenberger, T. (2025). Code Quality Defects in Introductory Programming. *ACM Transactions on Computing Education*, 25(4). <https://doi.org/10.1145/3749996>

- Recker, J., Indulska, M., Green, P. F., Burton-Jones, A., & Weber, R. (2019). Information systems as representations: A review of the theory and evidence. *Journal of the Association for Information Systems*, 20(6), 735–786. <https://doi.org/10.17705/1jais.00550>
- Reeves, B., Sarsa, S., Prather, J., Denny, P., Becker, B. A., Hellas, A., Kimmel, B., Powell, G., & Leinonen, J. (2023). Evaluating the Performance of Code Generation Models for Solving Parsons Problems With Small Prompt Variations. *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2023*, 299–305. <https://doi.org/10.1145/3587102.3588805>
- Wand, Y., & Weber, R. (1988). An Ontological Analysis of Some Fundamentals Information Systems Concepts. *Journal of the Association for Information Systems*, 35. <http://aisel.aisnet.org/icis1988/35>